

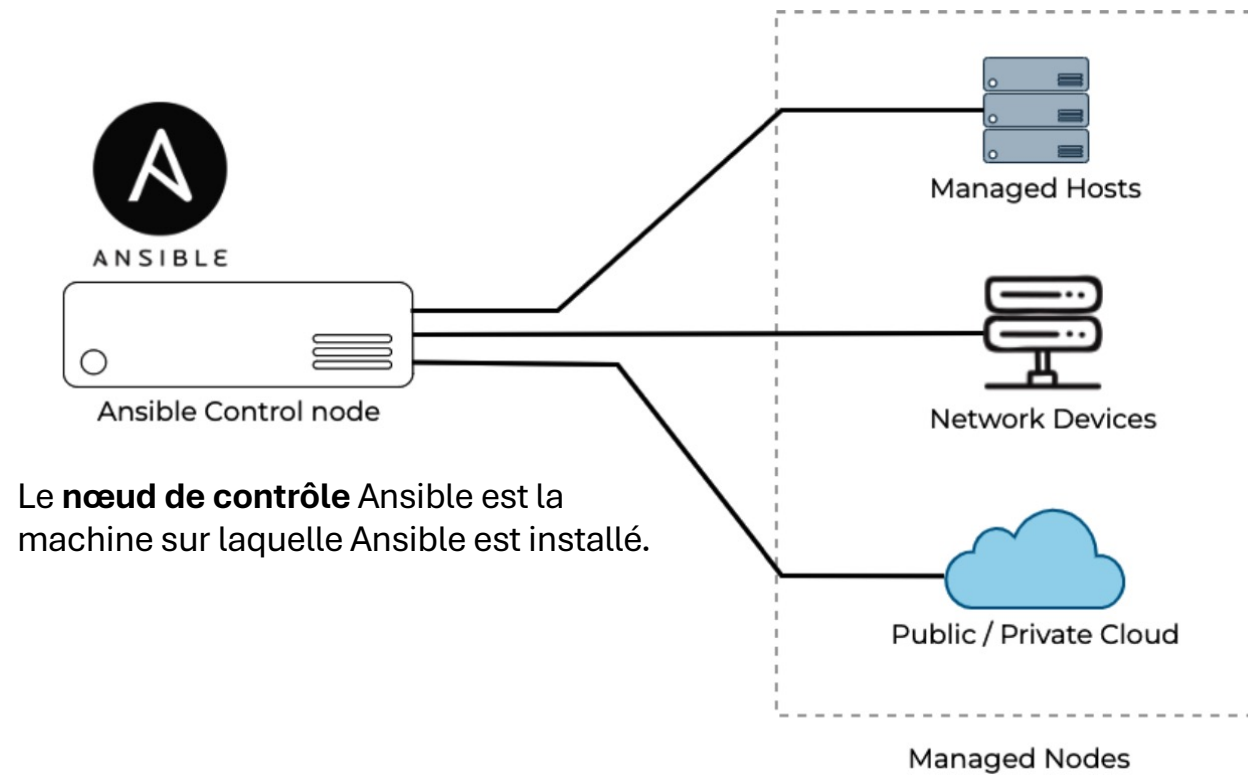


Présentation

Configuration Management (CM) => gestionnaire de configuration

=> déployer des configurations à travers un parc informatique (infrastructure)

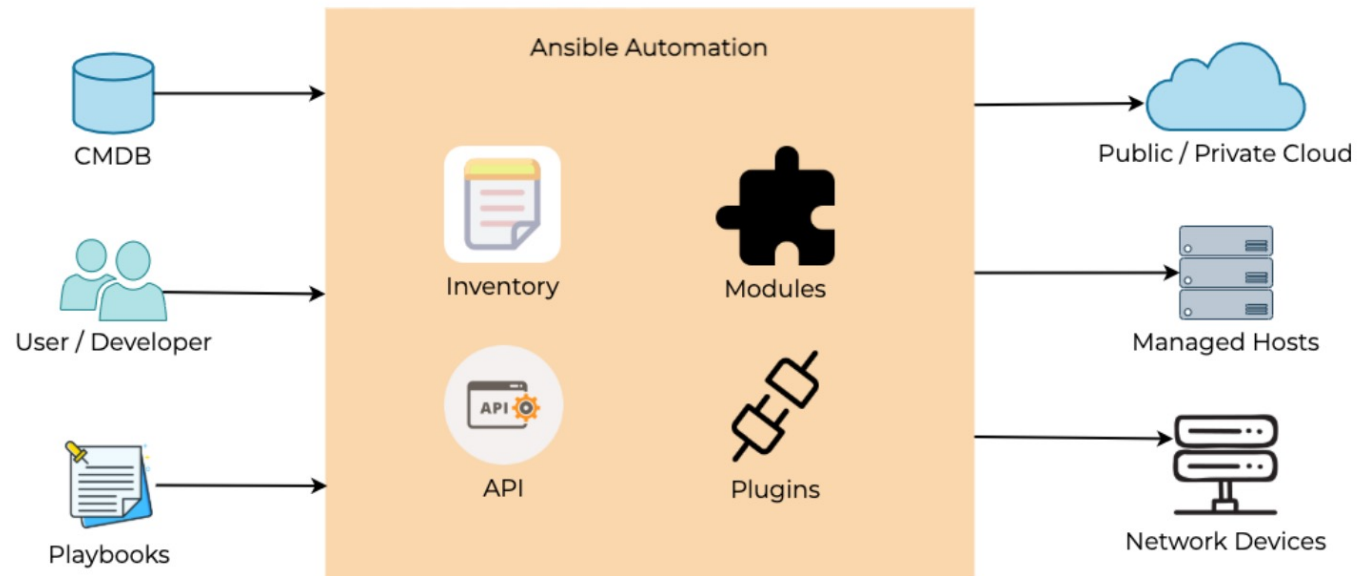
Composants



Les machines ou appareils que vous souhaitez automatiser sont appelés **hôtes gérés**.

- **Ansible est sans agent** => pas besoin d'installer d'agents sur les nœuds gérés.
- pour communiquer avec les hôtes gérés => ssh, WinRM (Windows), ...

Architecture



Ansible – Commande Ansible Ad-hoc

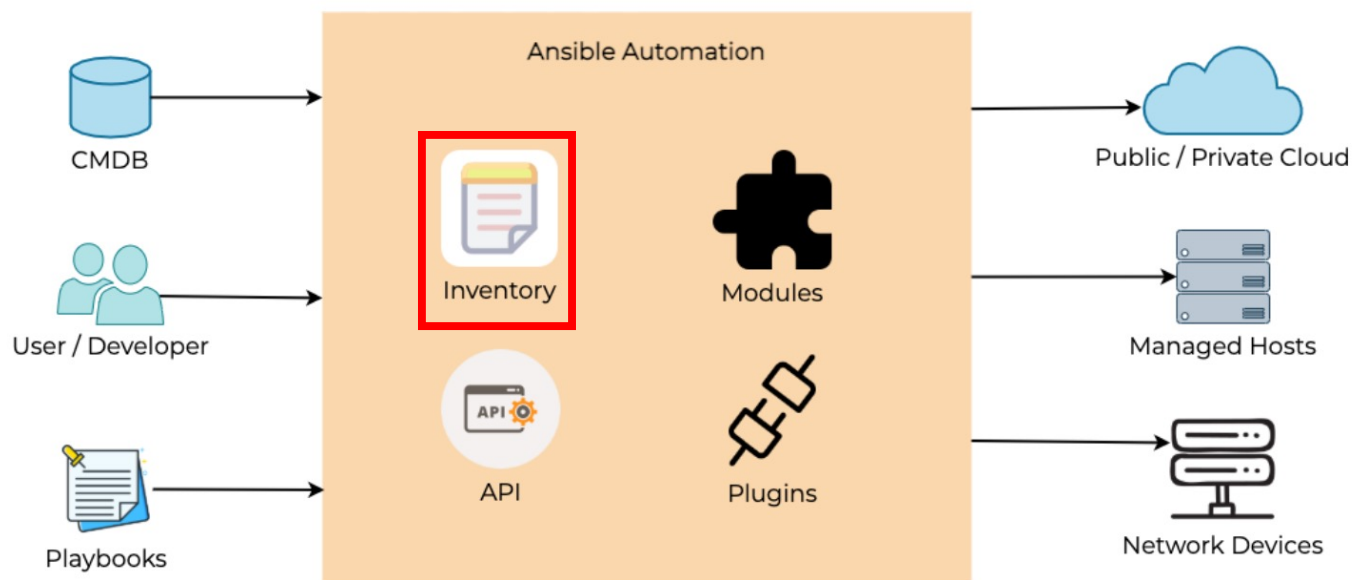
Dans Ansible, une **commande ad-hoc** est une **commande ponctuelle**, exécutée directement depuis la ligne de commande.

La syntaxe d'une commande ad-hoc Ansible est la suivante :

```
ansible <target> -i <inventory> -m <module> -a "<arguments>"
```

- **<target>** : l'hôte ou le groupe (défini dans l'inventaire) sur lequel vous souhaitez exécuter la commande
- **<inventory>** : Chemin d'accès au fichier d'inventaire
- **-m <module>** : le module Ansible à utiliser
- **-a <arguments>** les arguments spécifiques au module

Inventory



Ansible - Inventaire

L'inventaire Ansible est un fichier qui fournit des informations détaillées sur les nœuds gérés, notamment le nom d'hôte, les méthodes de connexion et les informations d'identification.

Format INI

```
[webserver]  
172.19.0.3
```

Format YAML

```
all:  
  hosts:  
    webserver:  
      ansible_host: 10.5.0.3
```

Tester l'inventaire

```
ansible webserver -i inventory.yml -m ping
```

Le module `ping` sert à tester :

- la connexion SSH
- l'accès à Python sur la machine distante
- la capacité d'Ansible à exécuter des commandes

Ansible – Inventaire - Groupes

Groupe : permet de regrouper des hôtes dans une même entité.

all : c'est le groupe par défaut qui comprend tous les hôtes.

Création d'un groupe « routers » et un groupe « switches ».

```
web_servers:  
  hosts:  
    10.5.0.3  
db_servers:  
  hosts:  
    10.5.0.4
```

Tester l'inventaire :

```
ansible web_servers -i inventory_grp.yml -m ping
```

```
ansible all -i inventory_grp.yml -m ping
```

Inventaire - Sous-groupes ou groupes enfants

```
web_servers:  
  hosts:  
    10.5.0.3  
db_servers:  
  hosts:  
    10.5.0.4  
servers:  
  children:  
    web_servers:  
    db_servers:
```

Un **sous-groupe** dans l'inventaire permet d'organiser vos hôtes en groupes hiérarchisés.

On peut créer des *groupes* contenant d'autres *groupes*, ce qui facilite la gestion d'environnements complexes (production, web, db, etc.).

Inventaire - Sous-groupes ou groupes enfants

La commande `ansible-inventory` vous fournit des détails sur vos fichiers d'inventaire d'hôtes. Elle peut être utile pour comprendre comment vos hôtes sont regroupés.

```
/shared # ansible-inventory -i inventory_grp.yml --graph
@all:
```

```
  |--@ungrouped:
  |--@web_servers:
  |   |--10.5.0.3
  |--@db_servers:
  |   |--10.5.0.4
```

```
/shared # ansible-inventory -i inventory_ssgroup.yml --graph
@all:
```

```
  |--@ungrouped:
  |--@servers:
  |   |--@web_servers:
  |   |   |--10.5.0.3
  |   |--@db_servers:
  |   |   |--10.5.0.4
```

Inventaire - Sous-groupes ou groupes enfants

Exercice

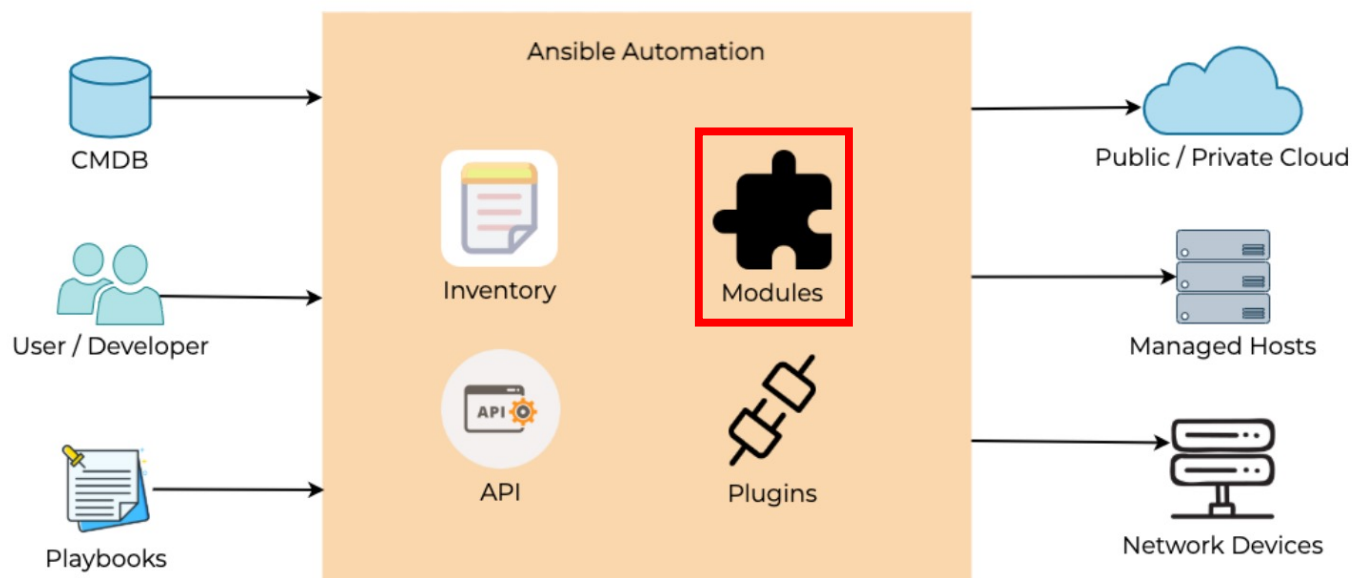
Ecrivez un fichier d'inventaire qui organise vos hôtes selon la structure suivante :

```
@all:
|--@ungrouped:
|--@web_servers:
| |--10.5.0.3
|--@db_servers:
| |--10.5.0.4
|--@servers:
| |--@web_servers:
| | |--10.5.0.3
| |--@db_servers:
| | |--10.5.0.4
```

Inventaire – Variables d'hôte et de groupe

```
web_servers:  
  hosts:  
    web1:  
      ansible_host: 10.5.0.3  
      http_port: 8080 # variable d'hôte  
    web2:  
      ansible_host: 10.5.0.6  
      http_port: 80 # variable d'hôte  
  vars:  
    web_packages: nginx # variable de groupe  
db_servers:  
  hosts:  
    10.5.0.4
```

Modules



Un module Ansible est un script réutilisable et autonome qui peut être utilisé pour réaliser certaines tâches spécifiques.

Les modules fournissent une interface définie avec des options permettant d'accepter des arguments et de renvoyer des informations à Ansible au format JSON. Lorsque vous exécutez une tâche à l'aide d'un module, le script du module sera exécuté sur la machine cible à l'aide de Python ou de PowerShell pour les machines Windows.

Les modules packaging :

- **apt** : outils de gestion des paquets dans Debian et Ubuntu.
- **Yum** : outils de gestion des paquets dans Fedora, CentOS et RedHat.

Les modules system :

- **service** : gestion des services.

Les modules réseau :

- Cisco (ios, nxos, aci, meraki, asa)
- Juniper
- Aruba
- Dell
- Arista
- BigIP F5
- Checkpoint
- Fortinet

https://docs.ansible.com/projects/ansible/2.9/modules/list_of_all_modules.html

Documentation

Galaxy User Guide

Galaxy Developer Guide

REFERENCE & APPENDICES

Module Index

All modules

Cloud modules

Clustering modules

Commands modules

Crypto modules

Database modules

Files modules

Identity modules

Inventory modules

Messaging modules

Monitoring modules

- [panos_match_rule](#) – Test for match against a security rule on PAN-OS devices or Pano
- [panos_mgtconfig](#) – configure management settings of device (D)
- [panos_nat_rule](#) – create a policy NAT rule (D)
- [panos_object](#) – create/read/update/delete object in PAN-OS or Panorama (D)
- [panos_op](#) – execute arbitrary OP commands on PANW devices (e.g. show interface all)
- [panos_pg](#) – create a security profiles group (D)
- [panos_query_rules](#) – PANOS module that allows search for security rules in PANW NC
- [panos_restart](#) – restart a device (D)
- [panos_sag](#) – Create a static address group (D)
- [panos_security_rule](#) – Create security rule policy on PAN-OS devices or Panorama ma
- [panos_set](#) – Execute arbitrary commands on a PAN-OS device using XPath and elemer
- [parted](#) – Configure block device partitions
- [patch](#) – Apply patch files using the GNU patch tool
- [pause](#) – Pause playbook execution
- [pear](#) – Manage pear/pecl packages
- [pids](#) – Retrieves process IDs list if the process is running otherwise return empty list
- [ping](#) – Try to connect to host, verify a usable python and return pong on success
- [pingdom](#) – Pause/unpause Pingdom alerts
- [pip](#) – Manages Python library dependencies

```
ansible-doc -l
```

Cette commande permet de lister l'ensemble des modules disponibles.

```
ansible-doc apt
```

Cette commande permet d'afficher l'aide et des exemples sur l'utilisation du module yum.

Exemple :

Pour connaître la **distribution Linux** que vous utilisez => gestionnaire de paquets : apk

```
cat /etc/os-release
```

Pour installer le paquet apache :

```
ansible web_servers -i inventory_grp.yml -m apk -a "name=apache2 state=present"
```

Le paquet est installé dans le dossier : /usr/sbin/

Module package

Ce module est conçu pour installer des paquets **quelle que soit la distribution Linux**.

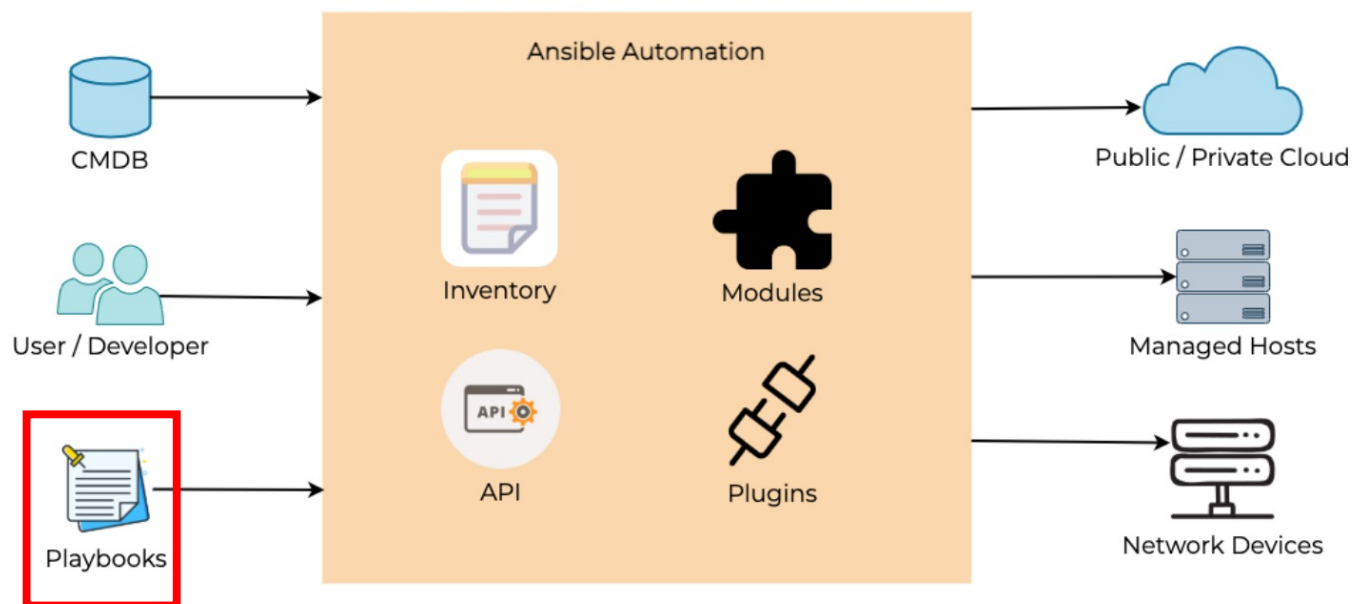
Il choisit automatiquement le bon gestionnaire selon la distribution :

Distribution	Gestionnaire utilisé
Debian / Ubuntu	apt
CentOS / Rocky / Alma / RHEL	yum ou dnf
Fedora	dnf
SUSE	zypper
Alpine	apk
Arch	pacman

Exercices :

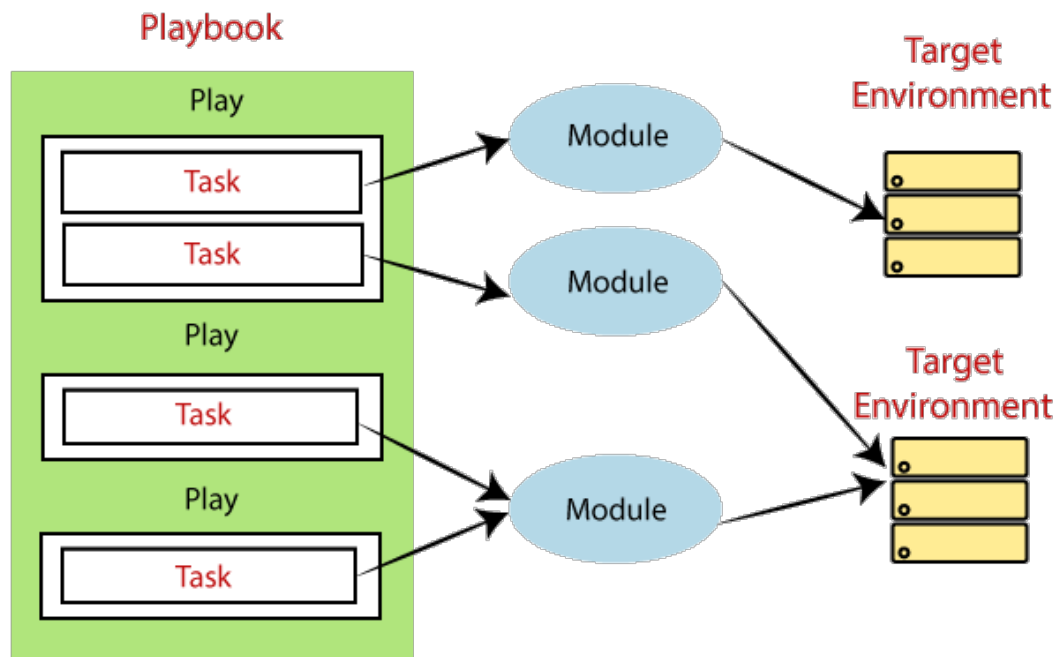
1. Démarrer avec Ansible le service « apache2 » sur le serveur « web ». Tester le serveur web avec l'outil « curl ».
2. Installer et démarrer le service « mariadb » sur le serveur « db ».

Playbook



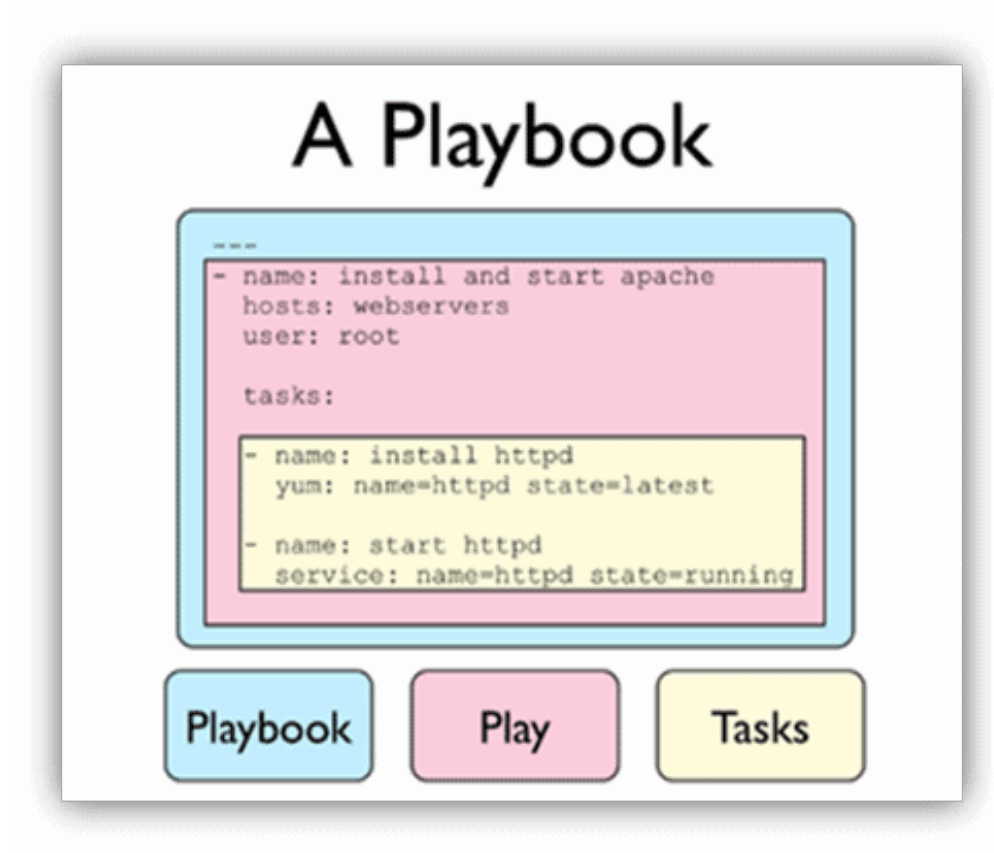
Playbook-Introduction

Un **playbook Ansible** est un fichier écrit en **YAML** qui permet de définir **une ou plusieurs tâches à exécuter sur des hôtes distants**, de façon **ordonnée, répétable et automatisée**.



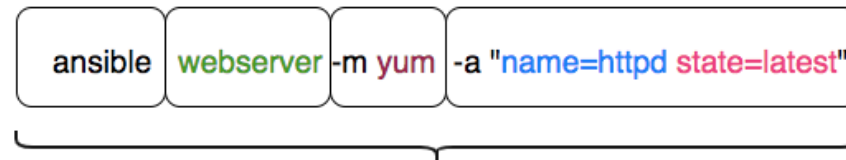
<https://www.javatpoint.com/ansible-playbooks>

Playbook-Structure



Playbook - Passage commande vers playbook

AD HOC command



Ansible Playbook

```
---  
- name: playbook name  
  hosts: webserver  
  tasks:  
    - name: name of the task  
      yum:  
        name: httpd  
        state: latest
```

www.middlewareinventory.com

Playbook - Passage commande vers playbook

1) Arrêter le service Apache

```
ansible web_servers -i inventory_grp.yml -m service -a "name=apache2 state=stopped"
```

2) Supprimer le service

```
ansible web_servers -i inventory_grp.yml -m apk -a "name=apache2 state=absent"
```

Playbook - Exemple

playbook_base.yml

```
- hosts: web_servers
  tasks:
    - name: Install apache2
      package:
        name: apache2
        state: present
    - name: Start the apache2 server
      service:
        name: apache2
        state: started
```

La commande pour exécuter le playbook :

```
ansible-playbook -i inventory_grp.yml playbook_base.yml
```

Playbook - Variables

```
- hosts: web_servers
  vars:
    apache_package: apache2
    apache_service: apache2
  tasks:
    - name: Install Apache
      package:
        name: "{{ apache_package }}"
        state: present
    - name: Start the Apache server
      service:
        name: "{{ apache_service }}"
        state: started
```

Déclaration des variables

Utilisation d'une variable

Playbook – Template (Jinja2)

```
- hosts: web_servers
vars:
  server_name: "web"
  message: "Bienvenue au DST"
tasks:
  - name: Install apache2
    package:
      name: apache2
      state: present
  - name: Start the apache2
    service:
      name: apache2
      state: started
```

Un **template** est un fichier texte (souvent au format **Jinja2**) utilisé pour générer **des fichiers de configuration dynamiques** sur les hôtes distants.

```
- name: Add the index.html
template:
  src: templates/index.html.j2
  dest: /var/www/localhost/htdocs/index.html
```

Templates/index.html.j2

```
<h1> {{ server_name }} </h1>
<h1> {{ message }} </h1>
```

Playbook – Handlers/notify

Les **handlers** et **notify** sont utilisés pour exécuter des tâches **uniquement lorsqu'un changement est détecté**, ce qui est particulièrement utile pour **redémarrer des services après une modification de configuration**.

```
- name: Template a file to /etc/file.conf
  template:
    src: templates/httpd.conf.j2
    dest: /etc/apache2/httpd.conf
  notify:
    - Restart the apache server
handlers:
  - name: Restart the apache server
    shell: pkill apache2 ; httpd -D FOREGROUND
```